

ShieldSense – Week 3 Research

Scam Message Characteristics and Application to ShieldSense

Student: Bijay Bahadur Chhetri

Role: Integration, Documentation & QA Lead

1. Introduction

ShieldSense is a proof-of-concept application designed to screen short SMS or chat messages for possible scam indicators and classify them into low, medium or high risk. In Week 3, my research focused on understanding how scammers construct messages and how these characteristics can inform the team's planned screening approach.

2. How Scammers Construct Messages

Scamwatch identifies three common tactics used by scammers: impersonation, urgency and emotion. Impersonation involves pretending to be a trusted organisation or person. Urgency pressures the recipient to act quickly, for example by creating a false deadline or account problem. Emotion uses feelings such as fear, excitement or hope to encourage a quick response. Scam messages can also request payment, personal information, login details, or ask the recipient to click a link or call a number. [1][2][3]

Scam characteristic	Example in a message	ShieldSense consideration
Impersonation	Pretending to be a bank, company or government agency	Check for trusted-organisation claims
Urgency	"Act now" or "limited positions available"	Identify pressure to act quickly
Emotion/reward	Fear of losing an account or promise of easy money	Identify emotional or reward-based language
Suspicious action	Click a link, call a number or provide details	Flag requests for links, payments or personal information

These characteristics will help the team develop the data labels, classification approach and user-facing explanations for ShieldSense.

3. Examples Relevant to ShieldSense

Example	Observed characteristics	Possible ShieldSense indicators
Fake employment message	Claims to offer easy, well-paid remote work, gives limited positions and asks the recipient to respond.	Impersonation, reward/high income, urgency, unusual recruitment request
Fake payment message	Claims a payment or transaction is pending and encourages the recipient to contact a number.	Impersonation, financial/payment context, urgency, unexpected transaction
Fake parcel delivery message	Claims a parcel could not be delivered or needs an update and asks the recipient to use a link.	Impersonation, urgency, suspicious link, personal-information request
Fake bank security message	Claims there is suspicious activity or a payment problem and asks the recipient to call a number or provide security details.	Impersonation, account issue, urgency, financial/credential request

These examples show why a message should not be classified from one word or feature alone. Several indicators can occur together, such as impersonation combined with urgency, a suspicious link, or a request for payment or personal information. [1][2]

4. Sender Information

Scamwatch also explains that business and organisation SMS messages should use branded sender IDs from 1 July 2026. Unregistered sender IDs may appear as "Unverified". An unverified sender should be treated with caution, but it should not automatically be treated as a scam. This is useful for ShieldSense because sender information can be considered as supporting context rather than a single decision rule. [1]

5. Application to ShieldSense

The research can be translated into a simple screening process: the message is entered into ShieldSense, possible scam indicators are analysed, a risk category is produced, and the interface provides an accessible Green, Amber or Red trust cue with an explanation. The team can use the researched indicators when preparing data, developing the model and checking the interface.

At this stage, these are planned features and research findings rather than completed system results. The project scope is a proof of concept using public or synthetic data and short English text. Real personal information should not be included in the dataset.

6. My Role – Integration, Documentation & QA

As the Integration, Documentation & QA Lead, I will focus on making sure the outputs from the data, model and interface work can be connected consistently. I will also help maintain project documentation and prepare checks for important behaviours, such as whether the displayed risk level matches the model output and whether the explanation shown to the user is clear and consistent.

7. Conclusion

Week 3 research improved my understanding of the patterns used in scam messages and how these patterns can support ShieldSense. The main findings are that impersonation, urgency and emotional pressure are important characteristics, while payment requests, suspicious actions and unexpected account or employment messages can provide additional context. These findings will guide the team's next stages of data preparation, model work, interface development and integration testing.

References

- [1] Scamwatch (2026), 'Text or SMS scams', Australian Competition and Consumer Commission.
- [2] Scamwatch (2026), 'Methods scammers use', Australian Competition and Consumer Commission.
- [3] Scamwatch (2026), 'Phishing scams', Australian Competition and Consumer Commission.
- [4] Scamwatch (2026), 'Scam alert: Job recruitment scams', 20 May 2026.
- [5] Scamwatch (2026), 'Scam alert: Fake purchase callback scams', 14 July 2026.
- [6] Australian Catholic University (2026), 'Student Placement Plan: ShieldSense'.

8. Coding Implementation and Evidence

Week 3 coding work completed: The ShieldSense repository was created in GitHub and opened in VS Code. A Python data-generation script (`create_dataset.py`) was developed to create a synthetic dataset of 200 short messages. The script assigns message categories and Green/Amber/Red risk levels, stores the messages in CSV format, and marks the records as `needs_review` until content verification is completed.

A second Python script (`verify_dataset.py`) was developed as a quality-check step. It checks the required columns, duplicate messages, empty messages, category counts, risk-level counts and verification status. This provides reproducible evidence for the data-preparation and QA process.

Implementation check	Result
Synthetic messages generated	200
Unique messages	200
Duplicate messages	0
Empty messages	0
Categories	7 scam/legitimate categories
Risk levels	120 Red, 20 Amber, 60 Green
Verification status	200 records still need content review

The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays the project structure: `SHEILDSSENSE` (folder), `data` (folder), `shieldsense_messages.csv` (file), `src` (folder), `create_dataset.py` (file), `verify_dataset.py` (file), `.gitattributes` (file), `.gitignore` (file), and `README.md` (file). The main editor shows the `create_dataset.py` script. The script defines a list of categories and risk levels, generates 200 synthetic messages, and writes them to a CSV file. The output of the script is displayed in the terminal window at the bottom.

```
src > create_dataset.py > ...
316     "category",
317     "risk_level",
318     "synthetic",
319     "verification_status"
320 ]
321
322 writer = csv.DictWriter(file, fieldnames=fieldnames)
323
324 writer.writeheader()
325 writer.writerows(records)
326
327
328 print("=== ShieldSense Dataset Generation ===")
329 print(f"Dataset created successfully: {OUTPUT_FILE}")
330 print(f"Total messages: {len(records)}")
331 print(f"Unique messages: {len(set(r['message'] for r in records))}")
332 print(f"Verification status: needs_review")
```

Figure 1. VS Code evidence showing the ShieldSense Python dataset-generation implementation and project structure.

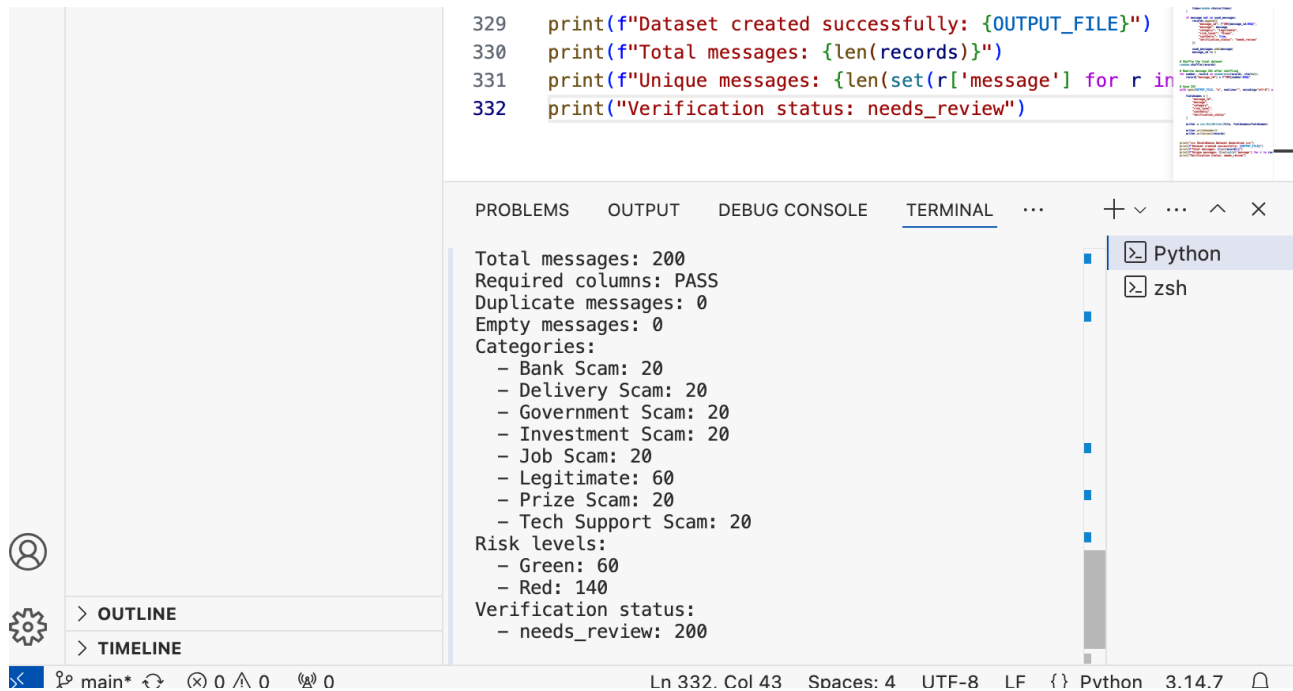
The screenshot shows the VS Code terminal window. The terminal output displays the results of running the `create_dataset.py` script. The output confirms that the dataset was created successfully, with 200 total messages and 200 unique messages. The verification status is `needs_review`. The terminal also shows the command used to run the script: `python3 src/create_dataset.py`.

```
Dataset created successfully: /Users/bijay_chhetri/Documents/GitHub/Sheildsense/data/shieldsense_messages.csv
Total messages: 200
Unique messages: 200
Verification status: needs_review
(base) bijay_chhetri@Bijays-MacBook-Air-2 Sheildsense %
python3 src/create_dataset.py
=== ShieldSense Dataset Generation ===
Dataset created successfully: /Users/bijay_chhetri/Documents/GitHub/Sheildsense/data/shieldsense_messages.csv
Total messages: 200
Unique messages: 200
Verification status: needs_review
(base) bijay_chhetri@Bijays-MacBook-Air-2 Sheildsense %
```

Figure 2. Terminal evidence showing successful generation of 200 messages and 200 unique messages.

9. Dataset Verification Evidence

The verification script was run after regeneration of the dataset. The output confirms that the required columns are present, there are no duplicate or empty messages, and the dataset contains the intended Green, Amber and Red risk levels. The records remain marked needs_review because structural checks do not replace human/content review.



The screenshot shows a code editor with a Python script and its terminal output. The script, located in `verify_dataset.py`, contains the following lines:

```
329 print(f"Dataset created successfully: {OUTPUT_FILE}")
330 print(f"Total messages: {len(records)}")
331 print(f"Unique messages: {len(set(r['message'] for r in records))}")
332 print("Verification status: needs_review")
```

The terminal output displays the results of the script execution:

```
Total messages: 200
Required columns: PASS
Duplicate messages: 0
Empty messages: 0
Categories:
- Bank Scam: 20
- Delivery Scam: 20
- Government Scam: 20
- Investment Scam: 20
- Job Scam: 20
- Legitimate: 60
- Prize Scam: 20
- Tech Support Scam: 20
Risk levels:
- Green: 60
- Red: 140
Verification status:
- needs_review: 200
```

The interface also shows a sidebar with 'OUTLINE' and 'TIMELINE' sections, and a status bar at the bottom indicating the file is `main*` at line 332, column 43, using Python 3.14.7.

Figure 3. Terminal evidence from `verify_dataset.py`: 200 messages, 0 duplicates, 0 empty messages, and the reported category/risk-level distribution.

Current status: The data-generation and automated QA checks are complete. The next implementation step is to build and run the rules-based ShieldSense baseline; that work is not claimed as completed in this Week 3 evidence.